ADVANCED ML COURSE PROJECT

Drift-Aware Explainable Social Bot Detection Using SHAP-Based Behavioral Drift Monitoring

Controlled benchmark + explainable drift characterization + adaptation-cost comparison for social-platform behavioral evolution

Updated: 2 Sep 2026 | Dataset: BotDriftBench v0.1 > v0.2 | Stack: Python 3.12 / XGBoost / SHAP

TASKS DONE	CURRENT PHASE	SEEDS	REMAINING
12 / 13	Calibration	10	5 items

How the Project Evolved

Five rounds of critical review reshaped the project from 'we built a dataset' into a proper contribution. Each round corrected trajectory before too much work went in the wrong direction.

Round 1 - Foundation Reset

Dataset must have true longitudinal trajectories (Markov carry-forward, not resampled random vectors). Humans must evolve too. Three-way drift comparison (feature vs prediction vs SHAP). Fit distributions to real public data. Start at 1K accounts.

Round 2 - Metric Rigor

Stop fighting AUC. Formalize SHAP attribution as Precision@k / Recall@k. Run drift delay across magnitudes x seeds. The novelty is the experimental structure, not 'first dataset.'

Round 3 - Literature Grounding

Don't overclaim 'drift detection' from prediction-drift results. Don't claim 'first adaptive bot detector' - EvoBot, BotEvo, continual botnet detection already exist. Our lane: controlled benchmark + explainability + adaptation-cost comparison.

Round 4 - Claims Calibration

Frame as efficiency, not superiority. Add recovery quality metrics (recovery_time, recovery_magnitude, adapt_efficiency). 10 seeds. Don't say 'causal' - say 'consistent with decision-boundary staleness.'

Round 5 - Real-Data Pivot

Freeze adaptation methodology. Start Cresci-2017 calibration. Pipeline: extract features > compare distributions > calibrate v0.2. Dependency drift next. GRU later. Dashboard last.

Completed Work

- Project plan adjusted for no-API constraints — DONE
- Drift injection dataset schema (P_t(X|Y) formalization, 5 drift scenarios) — DONE
- BotDriftBench v0.1 generator (true Markov trajectories, 30 features) — DONE
- Generated & validated v0.1 (1K accounts, 10 windows) — DONE
- Dataset documentation (corrected novelty framing) — DONE
- Rebuilt generator with true longitudinal trajectories — DONE
- Validated v0.1 against 6 critical questions — DONE
- Added cross-feature correlations & reduced separability — DONE
- Generated multi-magnitude benchmark variants (0.25x-2.0x) — DONE
- Re-validated against 6 questions with harder dataset — DONE

- Core: Static vs Periodic vs Drift-Aware (10 seeds, recovery metrics) — DONE
- Threshold sensitivity: 6 multipliers, 2.0x optimal — DONE
- Cresci-2017 real-data calibration (6.6M tweets) — RUNNING

Key Results: Adaptation Experiment

10 seeds x 4 magnitudes x 3 strategies. The headline: drift-aware achieves comparable robustness to periodic retraining while requiring substantially fewer retraining events.

Magnitude	Strategy	F1 Drop	Retrains	Recovery	Efficiency
0.25x	Static	0.034+/-0.005	0	0.37	--
	Periodic	0.021+/-0.001	4.0	0.53	0.006
	Drift-Aware	0.027+/-0.005	1.2	0.41	0.017
1.0x	Static	0.047+/-0.011	0	0.23	--
	Periodic	0.022+/-0.002	4.0	0.49	0.006
	Drift-Aware	0.028+/-0.005	1.8	0.41	0.012
2.0x	Static	0.075+/-0.013	0	0.15	--
	Periodic	0.020+/-0.002	4.0	0.52	0.005
	Drift-Aware	0.030+/-0.004	2.1	0.32	0.006

Core finding: Drift-aware uses 1-3 retrains vs periodic's fixed 4, achieving 2-3x better efficiency (F1 recovered per retrain). F1 recovery is slightly lower than periodic, consistent with decision-boundary staleness rather than increased class overlap. Zero false alarms at 2.0x threshold.

Cresci-2017 vs Synthetic v0.1 (Preliminary)

KS distances between real Cresci-2017 distributions and BotDriftBench v0.1 window 0. KS < 0.3 = acceptable match. Results are preliminary (pre-bugfix); corrected run in progress.

Feature	KS Human	KS Bot	Verdict
verified	0.008	0.000	Match
has_url	0.024	0.077	Match
default_profile_image	0.136	0.110	Match
weekend_ratio	0.140	0.145	Match
screen_name_length	0.163	0.249	Match
default_profile	0.220	0.176	Match
caps_ratio	0.274	0.274	Match
followers_count	0.352	0.267	Human mismatch
friends_count	0.419	0.129	Human mismatch
statuses_count	0.371	0.718	Mismatch
posting_regularity	0.633	0.763	Mismatch
content_similarity	0.963	0.899	Mismatch
burst_score	1.000	1.000	Mismatch

emoji_ratio	0.718	0.668	Mismatch
text_entropy	0.694	0.699	Mismatch
retweet_ratio	1.000*	0.998*	Bug - rerunning

** Bug: was using retweet_count (retweeted by others) instead of retweeted_status_id (is a retweet). Fixed in current run.*

Calibration targets for v0.2: 13 features need recalibration. Biggest gaps: content features (burst_score, content_similarity, emoji_ratio, text_entropy) and count features (statuses_count, posting_regularity). Binary features and temporal ratios match well.

Cresci-2017 Dataset Structure

Category	Label	Accounts	Tweets	Coverage
genuine_accounts	human	3,474	2,839,362	All 28 features
social_spambots_1	bot	991	1,610,034	All 28 features
social_spambots_2	bot	3,457	428,542	All 28 features
social_spambots_3	bot	464	1,418,557	All 28 features
traditional_spambots_1	bot	1,000	145,094	All 28 features
fake_followers	bot	3,351	196,027	All 28 features
traditional_spambots_2	bot	100	--	8 profile only
traditional_spambots_3	bot	403	--	8 profile only
traditional_spambots_4	bot	1,128	--	8 profile only
Total		14,368	6,637,616	

What's Next

1. Finish Cresci-2017 Calibration (in progress)

Analysis re-running with bug fixes: retweet_ratio uses retweeted_status_id, account_age_days handles timezones, category names strip .zip suffix. Output: corrected distributions, per-category profiles, correlation structure, separability matrix, archetype matching.

2. Build BotDriftBench v0.2 - Calibrated Generator

Use KS mismatch map to recalibrate the 13 mismatched features. Replace invented parameters with Cresci-2017 empirical distributions. Keep drift injection and trajectory model unchanged.

3. Rerun Frozen Adaptation Experiment on v0.2

Same methodology (10 seeds, 3 strategies, 4 magnitudes), same metrics. Only the input dataset changes. Validates whether the efficiency finding holds on calibrated data.

4. Feature Dependency / Correlation Drift

Fourth drift signal motivated by 'Bots Don't Sit Still' (Dec 2025): 153 feature pairs change correlation across bot generations. Implemented as correlation-matrix distance between windows.

5. GRU Temporal Baseline

Baseline temporal model. XGBoost is primary classifier; GRU shows framework is model-agnostic. Lower priority.

6. Streamlit Dashboard (last)

Demonstration layer only. Visualization of drift signals, SHAP attributions, adaptation events.

Standing Constraints

- No API access: No Twitter/X, Instagram, or Google APIs. All real data from public datasets (Cresci-2017 now; TwiBot-22 later if institutional access obtained).
- Frame as efficiency: 'Drift-aware achieves comparable robustness to periodic retraining while requiring substantially fewer retraining events.' Not superiority.
- No causal claims: Say 'consistent with decision-boundary staleness' not 'causal.' The controlled benchmark shows correlation structure, not causation.
- No priority claims: Don't claim 'first drift-aware adaptive bot detector.' EvoBot, BotEvo, continual botnet detection exist. Our lane: controlled benchmark + explainability + cost comparison.
- Cresci is calibration, not validation: Don't claim 'Cresci represents modern bot behavior.' It's 2017 data used to ground synthetic distributions.

Project Files

File	Purpose	Status
generate_dataset.py	BotDriftBench v0.1 generator	Stable
adaptation_experiment.py	Static vs Periodic vs Drift-Aware	Frozen
threshold_sensitivity.py	6 threshold multipliers	Frozen
real_data_analysis.py	Cresci-2017 feature extraction	Running
botdriftbench_v0.1.csv	1K accounts x 10 windows x 30 feat	Complete
adaptation_summary_v2.csv	120 rows: 10s x 4m x 3 strategies	Complete
threshold_sensitivity.csv	240 rows: 10s x 4m x 6 thresholds	Complete
cresci-2017.csv.zip	Raw Cresci-2017 (444 MB)	Downloaded
cresci2017_*.csv	Extracted features & comparisons	Rerunning